ARQUITECTURA

EN CAPAS, CLIENTE-SERVIDOR, MONOLÍTICA Y MICROSERVICIOS

EN CAPAS

Patrón de diseño de software que permite organizalo por medio de capaz horizontales.

- Cada capa cumple una **unica responsabilidad**
- Una capa solo puede comunicarse con la **capa inferior inmediata**

Resuelve problemas, por ejemplo:
- Código espaguetti: No **mezcla** reglas del negocio y BBDD
- Facilita el **mantenimiento** para que no sea complejo realizar cambios
- Permite trabajo en equipo **sin pisarse el código**.

Para aplicaciones medianas o grandes con lógica de negocio compleja

Ventajas
- **Mantenibilidad:** Pérmite identificar la parte del código que hay que cambiar
- **Ecalabilidad:** Se puede modificar una capa sin cambiar todo lo demás
- **Trabajo grupal:** Varios desarrolladores tabajan simultáneamente sin pisarse el código

Desventajas
- En **proyectos pequeños**, muchas capas pueden generar **código innecesario**, aumentan la cantidad de llamadas entre componentes y **más esfuerzo**

CLIENTE-SERVIDOR

Modelo en que las tareas se dividen entre los clientes y se solicitan servicios por medio de interfaces.

Proceso:
- El cliente envía una **petición** mediante la red
- El servidor **procesa la solicitud** y realiza según la **lógica del negocio**
- Realiza las **consultas** necesarias a la BBDD.
- Envía una **respuesta** al cliente

Resuelve problemas, por ejemplo:
- **Inconsistencia de datos**: la gestión dispersa e ineficiente de recursos, concentrándose en una única fuente.
- Elimina la necesidad de los usuarios tengan **dispositivos con gran capacidad de almacenamiento**.

Componentes
- **Cliente:** Dispositivo que actúa como la interfaz de usuario
- **Servidor:** Sistema central que espera las solicitudes del cliente
- **Red:** Infraestructura de comunicaciones que da un medio de transmisión físico
- **Protocolos:** Conjunto de reglas estandrizadas que determinan formato y la sintaxis

Permite la comunicación y el intercambio de información entre diferentes sistemas

MONOLÍTICA

Modelo en que los componentes principales de una aplicación están integrados.

Características:
- Todos los módulos son parte de la **misma aplicación**
- Las funciones comparten el **mismo código base**
- Cuando se actualiza la aplicación hay que **desplegarla nuevamente**

Formas de desplegar una aplicación monolítica:
- En un entorno virtual que tiene **recursos propios**
- La aplicación es **empaquetada** en un **contenedor**
- Se instala y se ejecuta en **servidores físicos**
- Se mantiene en una **plataforma en la nube**

MICROSERVICIOS

Estilo de desarrollo de software donde una aplicación se divide en pequeños servicios.

Características:
- Cada servicio se puede **desarrollar, desplegar y escalar** de manera autónoma
- Cada microservicio se centra en **una sola función o dominio** específico del negocio
- Cada servicio **gestiona su propia BBDD** sin compartir dependencias
- Los servicios interactúan entre sí por medio de **interfaces y peticiones estándar**

Para sistemas complejos y con alta demanda, cargas de tráfico desiguales entre servicios y equipos divididos

Ventajas
- **Escalabilidad granular**
- **Aislamiento de fallos**: si falla algo no se cae toda la pagina
- Ponerle un sentido a cada servicio y no se satura cada servidor (**libertad tecnológica**)

Desventajas
- Aumenta la dificultad en la **gestión de comunicaciones y redes**.
- Gestionar bases de datos distribuidas hace que **mantener datos sincronizados** sea más complicado.
- Dificulta la **realización de pruebas integradas** por la cantidad de componentes independientes.

Resuelve problemas, por ejemplo:
- Bloqueos en equipos grandes por código compartido
- Caídas totales por fallos en un solo componente
- Lentitud en despliegues al tener que actualizar todo el sistema